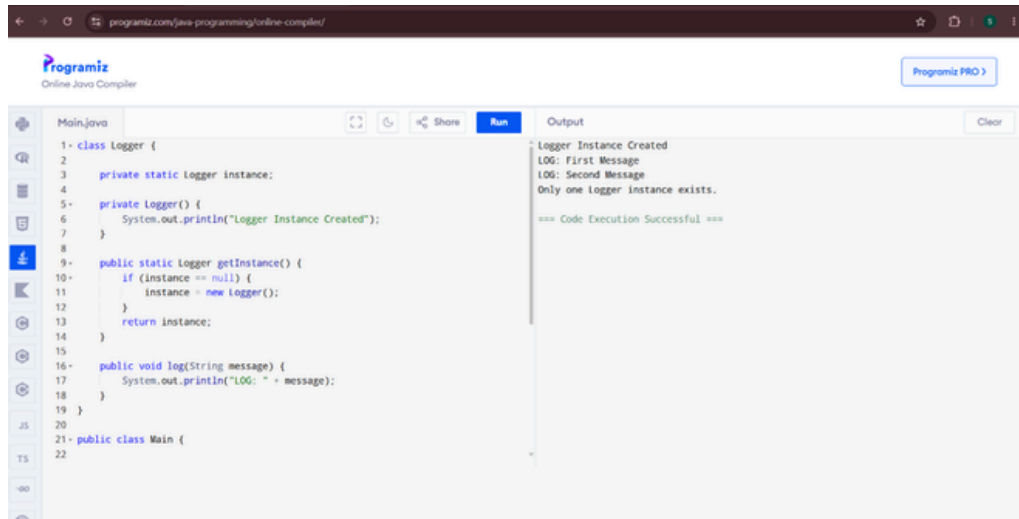


WEEK 1 ASSIGNMENT

DESIGN PATTERNS

1. Singleton Pattern Example (Logger)



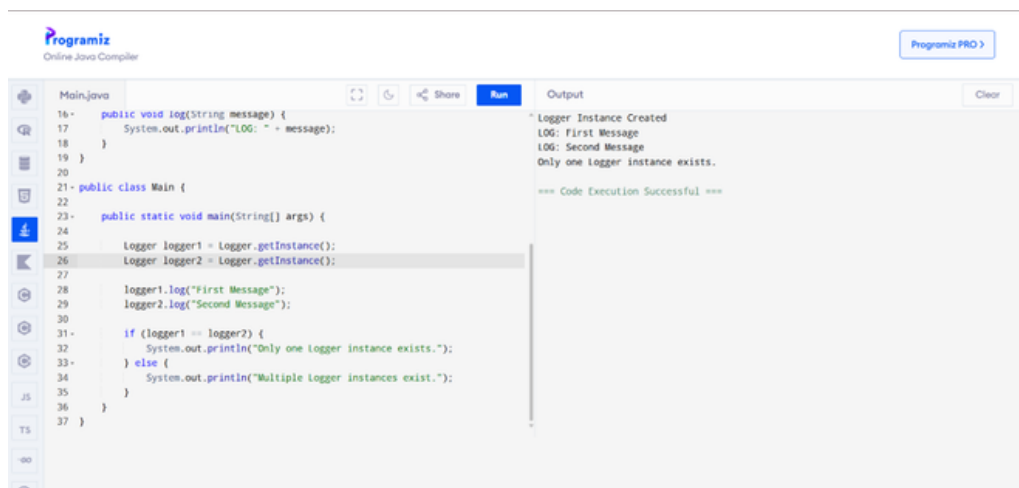
The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
1- class Logger {
2
3-     private static Logger instance;
4
5-     private Logger() {
6-         System.out.println("Logger Instance Created");
7-     }
8
9-     public static Logger getInstance() {
10-         if (instance == null) {
11-             instance = new Logger();
12-         }
13-         return instance;
14-     }
15
16-     public void log(String message) {
17-         System.out.println("LOG: " + message);
18-     }
19- }
20
21- public class Main {
22
```

The output window displays the following text:

```
Logger Instance Created
LOG: First Message
LOG: Second Message
Only one Logger instance exists.

=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

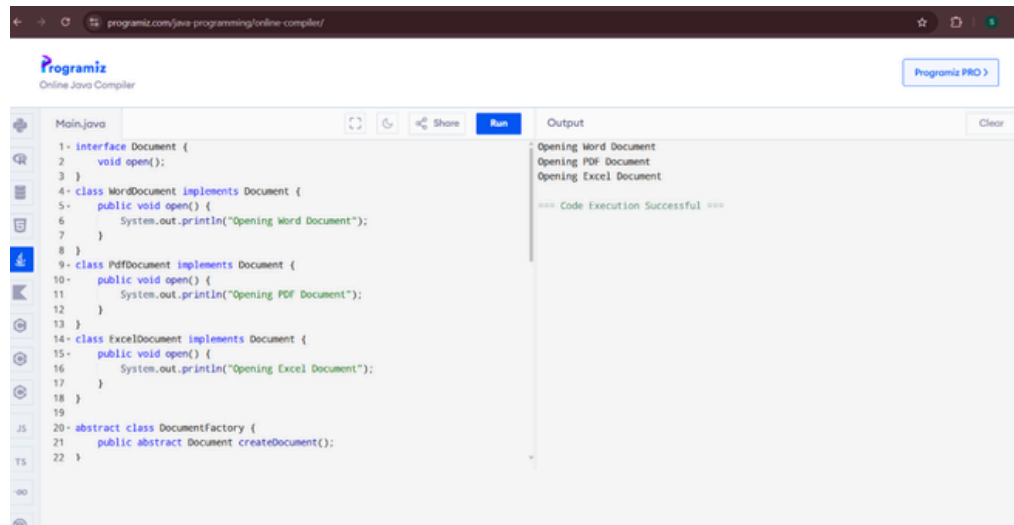
```
16-     public void log(String message) {
17-         System.out.println("LOG: " + message);
18-     }
19- }
20
21- public class Main {
22
23-     public static void main(String[] args) {
24-         Logger logger1 = Logger.getInstance();
25-         Logger logger2 = Logger.getInstance();
26-
27-         logger1.log("First Message");
28-         logger2.log("Second Message");
29-
30-         if (logger1 == logger2) {
31-             System.out.println("Only one Logger instance exists.");
32-         } else {
33-             System.out.println("Multiple Logger instances exist.");
34-         }
35-     }
36- }
37- }
```

The output window displays the following text:

```
Logger Instance Created
LOG: First Message
LOG: Second Message
Only one Logger instance exists.

=== Code Execution Successful ===
```

2. FactoryMethodPatternExample.java

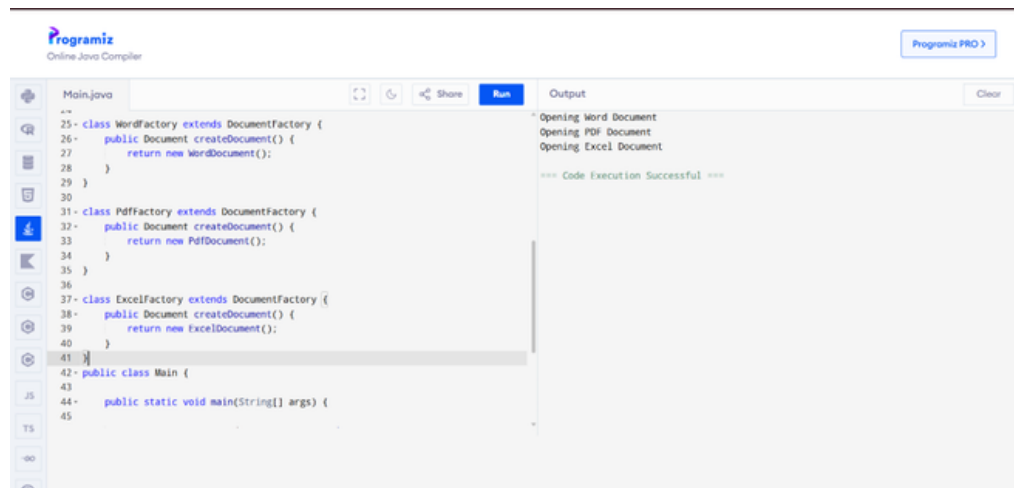


The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
1- interface Document {
2-     void open();
3- }
4- class WordDocument implements Document {
5-     public void open() {
6-         System.out.println("Opening Word Document");
7-     }
8- }
9- class PdfDocument implements Document {
10-    public void open() {
11-        System.out.println("Opening PDF Document");
12-    }
13- }
14- class ExcelDocument implements Document {
15-    public void open() {
16-        System.out.println("Opening Excel Document");
17-    }
18- }
19-
20- abstract class DocumentFactory {
21-    public abstract Document createDocument();
22- }
```

The output window shows the following text:

```
Opening Word Document
Opening PDF Document
Opening Excel Document
=== Code Execution Successful ===
```

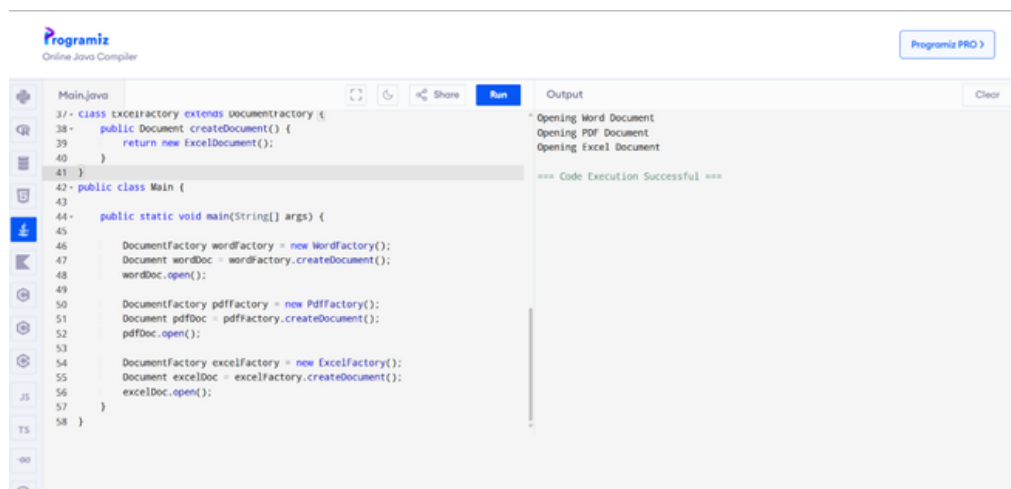


The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
25- class WordFactory extends DocumentFactory {
26-     public Document createDocument() {
27-         return new WordDocument();
28-     }
29- }
30-
31- class PdfFactory extends DocumentFactory {
32-     public Document createDocument() {
33-         return new PdfDocument();
34-     }
35- }
36-
37- class ExcelFactory extends DocumentFactory {
38-     public Document createDocument() {
39-         return new ExcelDocument();
40-     }
41- }
42-
43- public class Main {
44-     public static void main(String[] args) {
45-         // ... (code from previous screenshot) ...
46-     }
47- }
```

The output window shows the following text:

```
Opening Word Document
Opening PDF Document
Opening Excel Document
=== Code Execution Successful ===
```



The screenshot shows the Programiz Online Java Compiler interface. The code editor contains the following Java code:

```
37- class ExcelFactory extends DocumentFactory {
38-     public Document createDocument() {
39-         return new ExcelDocument();
40-     }
41- }
42-
43- public class Main {
44-     public static void main(String[] args) {
45-         DocumentFactory wordFactory = new WordFactory();
46-         Document wordDoc = wordFactory.createDocument();
47-         wordDoc.open();
48-
49-         DocumentFactory pdfFactory = new PdfFactory();
50-         Document pdfDoc = pdfFactory.createDocument();
51-         pdfDoc.open();
52-
53-         DocumentFactory excelFactory = new ExcelFactory();
54-         Document excelDoc = excelFactory.createDocument();
55-         excelDoc.open();
56-     }
57- }
58- }
```

The output window shows the following text:

```
Opening Word Document
Opening PDF Document
Opening Excel Document
=== Code Execution Successful ===
```